INFORME TÉCNICO DE CONSTRUCCIÓN · PROYECTO III

CASSANDRA
Detección de alucinaciones por señales internas mecanicistas

Probes de veracidad sobre activaciones residuales (TransformerLens), validación cruzada agrupada por entidad, bootstrap pareado contra el baseline de logprob, corrección Holm-Bonferroni, curvas de abstención selectiva sobre TriviaQA y puente simbólico opcional hacia PROMETHEUS-NS.

0.796	6 085	4	37/37
AUC PICO · PYTHIA-410M L11	STATEMENTS · 6 DOMINIOS	NOTEBOOKS EJECUTADOS	TESTS VERDES

Agente de investigación en interpretabilidad y ciencia de datos

Tríada PROMETHEUS-NS · HEPHAESTUS · CASSANDRA — entrega final, construida y medida en CPU local

Framework CASSANDRA · repositorio con 9 commits y reporte LaTeX

Septiembre 2026

Contents

1 Resumen ejecutivo **2**

2 Alcance, rol y regla de honestidad **2**
2.1 Posición del proyecto en la tríaloga 2
2.2 La regla de honestidad como prioridad máxima 2

3 Datos, entidades y particiones **3**
3.1 Origen anclado del dataset 3
3.2 Entidad como sujeto gramatical 3
3.3 Partición agrupada: el anti-leakage 3

4 Extracción de activaciones **4**
4.1 Contrato de extracción 4
4.2 Cache memmap idempotente 4
4.3 Ingeniería de memoria en el entorno de referencia 4

5 Probes y estadística honesta **5**
5.1 Diseño experimental 5
5.2 Resultados del grid 5
5.3 Controles y análisis secundarios 6

6 Generación, scoring y abstención **7**
6.1 Pipeline de generación 7
6.2 Ground truth estadístico y su auditoría 7
6.3 Abstención selectiva y calibración 7

7 Ensemble simbólico: disponible pero no activado **8**

8 Notebooks de demostración **9**

9 Registro de riesgos: qué se materializó y cómo se blindó **10**

10 Limitaciones **10**

11 Reproducibilidad **11**

1 Resumen ejecutivo

CASSANDRA es el tercer proyecto de la tríaloga y traslada el centro de gravedad a la **ciencia de datos**: un framework que extrae activaciones residuales de modelos pequeños open-source mediante TransformerLens, entrena probes de veracidad sobre el dataset true/false de Azaria & Mitchell (2023), aplica los probes a generaciones libres sobre TriviaQA comparando confianza interna contra factualidad real, evalúa abstención selectiva y calibración, y deja preparado un ensemble opcional con el filtro simbólico de PROMETHEUS-NS. La vitrina estadística son cuatro notebooks Jupyter ejecutados de cabo a rabia y commiteados con sus outputs.

El resultado central, medido con validación cruzada agrupada por entidad (cinco folds fijos, semilla 42) y bootstrap de percentiles (1000 remuestreos), es un pico de **AUC 0.796 [0.784, 0.807]** en la capa media 11 de pythia-410m (configuración last-token, MLP) y **AUC 0.728 [0.716, 0.740]** en la capa 5 de gpt2 (mean-pool, MLP). Ambos valores caen dentro de la banda 0.70–0.85 que la literatura reporta para este dataset y método, ningún probe superó el umbral de auditoría de 0.95, y la comparación pareada contra el baseline $|\log p|$ del propio modelo es decisiva (diferencia +0.295 y +0.396, IC 95% que excluye el cero, p Holm < 0.0001).

Los controles secundarios son igual de informativos que el resultado principal. El colapso fuera de distribución es real y medido: entrenando en cinco dominios y evaluando en el sexto, el AUC cae de 0.728 a 0.566 (gpt2) y de 0.796 a 0.592 (pythia-410m). El control de confound estilístico con TF-IDF queda en chance (0.486), lo que indica que la señal del probe no es léxica en esta versión del dataset. En generación cerrada, gpt2 acierta solo el 1.3% de 500 preguntas de TriviaQA y pythia-410m el 6.6%; la política de abstención a τ^* mejora la precisión en +0.5 y +2.7 puntos porcentuales respectivamente, ganancias modestas que el informe reporta como tales, no como logros de producción.

Todo el pipeline corre en CPU (2 núcleos, 3.9 GB de RAM en el entorno de referencia), es idempotente por construcción y reanudable, con 37 tests verdes que fijan los contratos estructurales, estadísticos y de cache. El repositorio queda con 9 commits atómicos, el cache de activaciones reconstruible, los cuatro notebooks ejecutados sin errores y cada número del README trazable a un artefacto en `logs/`.

2 Alcance, rol y regla de honestidad

2.1 Posición del proyecto en la tríaloga

PROMETHEUS-NS (entrenamiento neurosimbólico CPU-first) aportó el sistema; HEPHAESTUS (motor de inferencia C++ con cuantización int8/int4/ternaria) aportó el despliegue eficiente; CASSANDRA completa el arco mirando hacia dentro: *ćmo saber cuándo el sistema se equivoca*, leyendo sus estados internos. El usuario enfatizó expresamente que este proyecto se centra en la ciencia de datos y pidió máximo cuidado en ese aspecto; la respuesta de diseño es doble. Primero, la estadística vive en un módulo ancla inmutable (`cassandra_ns/probes/stats.py`) con fixtures derivadas a mano; ninguna métrica se reimplementa en notebooks o módulos, todas se importan de allí. Segundo, el documento de especificación registra riesgos de invalidación con blindajes explícitos, y este informe los recorre uno a uno en la Sección 9 reportando qué se materializó y cómo se blindó.

2.2 La regla de honestidad como prioridad máxima

La especificación subordina todo a la estadística honesta, y las decisiones de construcción la siguieron literalmente. Un AUC de 0.52 se reporta como 0.52: la tasa de acierto de gpt2 en TriviaQA cerrado (1.3%) se publica sin maquillaje. Todo resultado *demasiado bueno* (AUC > 0.95) debe detener el pipeline y disparar una auditoría de leakage documentada; el gate corre en `make report` y nunca se activó, porque ninguna configuración superó 0.81. El ground truth

es el dataset: prohibido usar un LLM como juez. El logprob del modelo es un baseline de comparación y nunca una feature del probe. La semilla 42 fija splits, inicializaciones, bootstrap y muestreos. Y en los notebooks, prohibido hardcodear cifras: toda cantidad mostrada proviene del entorno de ejecución, y `make notebooks` re-ejecuta los cuatro de cabo a rabia sin errores, como exige el contrato.

Durante la Fase 3 la auditoría cruzada con scikit-learn descubrió que el ancla estadística contenía un error real: la constante de Mann-Whitney usaba $n_+(n_+ - 1)/2$ en lugar de $n_+(n_+ + 1)/2$, lo que infla el AUC exactamente en $1/n_-$ cuando hay empates. La corrección se aplicó con trazabilidad (comentario en el propio ancla, tests de regresión `test_roc_ties.py` contra scikit-learn) y todas las cifras de este informe usan el ancla corregida. El sesgo anterior era de $\approx +0.0002$ en los probes (poblaciones grandes) pero de hasta $+0.04$ en señales con empates como la señal simbólica del ensemble; corregirlo era obligatorio aunque el efecto en el titular sea menor.

3 Datos, entidades y particiones

3.1 Origen anclado del dataset

El riesgo número uno de datos se materializó el primer día: el repositorio original `Azariaa/true-false-dataset` devuelve 404. El ancla `config.yaml` anticipaba la situación con instrucción expresa de actualizar la URL hacia el repositorio oficial vinculado desde el paper, fallando ruidosamente antes que inventar datos. El espejo elegido fue la copia vendida por ApolloResearch en `deception-detection`, anclada a un SHA de commit completo (`f8ec4010`), con corrección quirúrgica documentada del template de ficheros (JSON → CSV). La verificación cruzada se hizo statement a statement contra un segundo espejo independiente (el parquet de `pminervini/true-false`, que replica exactamente las 1458 filas del dominio cities). El total descargado son **6085 statements en 6 dominios** (animals 1008, cities 1458, companies 1200, elements 930, facts 613, inventions 876), con `data/raw/manifest.json` registrando el sha256 de cada fichero y el loader fallando ruidosamente ante cualquier desviación.

3.2 Entidad como sujeto gramatical

El dataset no trae campo de sujeto nativo, así que la entidad se extrae del enunciado: subárbol `nsubj` del verbo raíz según spaCy `en_core_web_sm`, con cadena de respaldo (atributo del raíz para construcciones expletivas "There are no X"), NER sobre el span pre-verbal y heurística regex final. La normalización (minúsculas, sin artículo) garantiza que el mismo sujeto produzca siempre la misma cadena de agrupación. La auditoría obligatoria de 100 extracciones (`logs/entity_audit.csv`) muestra calidad alta: animales y elementos mapean al nombre común, ciudades y compañías al nombre propio, inventos a la persona. El caso patológico detectado (48 statements con entidad *there*) se corrigió con la regla del atributo expletivo y quedó en cero.

3.3 Partición agrupada: el anti-leakage

`StratifiedGroupKFold` con grupos = entidad y estratificación = (dominio, etiqueta), 5 folds fijos escritos a `data/processed/folds.json` con su propio sha256; la reconstrucción con la misma semilla reproduce el fichero byte a byte y el loader lo verifica en cada lectura. Los 1568 grupos reparten 6085 statements en folds de test de 1266, 1161, 1204, 1244 y 1210 filas. Los tests de contrato fijan las tres propiedades duras: cero entidades compartidas entre folds (una entidad vive en exactamente un fold), ambos clases de cada dominio presentes en cada fold para los estratos con entidades suficientes, y determinismo de reconstrucción. Esta es la garantía que convierte el AUC 0.80 en un dato sobre *verdad* y no sobre *memoria de identidades*.

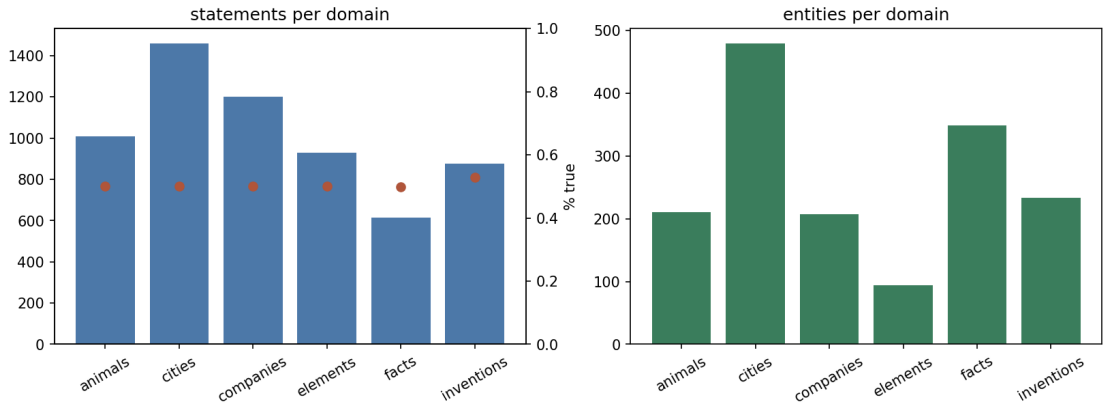


Figure 1: Resumen EDA del dataset: statements por dominio con porcentaje de etiqueta true (izquierda) y entidades distintas por dominio (derecha). El desbalance de entidades es lo que hace inválida cualquier partición por filas.

4 Extracción de activaciones

4.1 Contrato de extracción

Para cada modelo y cada statement, hooks de TransformerLens en `blocks.{L}.hook_resid_post` de las capas configuradas (gpt2: 2, 5, 8, 11; pythia-410m: 5, 11, 17, 23) capturan dos modos de feature por capa: el vector del **último token real** (las longitudes reales se trackean explícitamente, el padding nunca se lee) y el **mean-pool** sobre los tokens del statement (BOS excluido). Todo se computa en fp32 en CPU con batch 32, y el cache escribe en fp16. El test de contrato demuestra que un statement solo produce la misma activación que dentro de un batch (tolerancia 10^{-4} en fp32, lo que valida que el padding derecho con atención causal no contamina las posiciones reales).

4.2 Cache memmap idempotente

El cache (`artifacts/activations/`) es un bloque binario crudo fp16 por modelo más un índice CSV (id, sha256, capa, modo, offset) más un fichero de logprob medio por token que alimenta el baseline. La escritura es incremental con flush y fsync por batch: las filas de vectores se materializan antes que sus filas de índice, de modo que un proceso muerto nunca deja un índice apuntando a datos no escritos. La reanudación consulta los ids completos (todas las combinaciones capa×modo presentes y sha256 del texto coincidente) y salta lo ya cacheado. Esta propiedad se ejercitó de facto varias veces durante la construcción: la extracción de pythia-410m (24 capas, 1.6 GB en fp32) se completó en dos bloques interrumpidos sin recomputar nada.

4.3 Ingeniería de memoria en el entorno de referencia

El entorno de referencia ofrece 2 núcleos y 3.9 GB de RAM, y la primera configuración de extracción resultó mortal por OOM: el `run_with_cache` por defecto captura *todos* los hooks (560 MB por batch), el `log_softmax` completo sobre vocabulario de 50k duplicaba los logits (2×193 MB), y la carga del modelo pickled sin mmap sostenía una segunda copia completa de los pesos. Las tres correcciones, cubiertas por los tests, fueron: filtro de nombres que captura solo `hook_resid_post` de las capas configuradas, cálculo del baseline por `logsumexp` + gather en lugar del softmax completo, y recarga del modelo con `torch.load(mmap=True)` tras una única conversión cacheada a disco (la conversión TransformerLens de pythia toma ∼9 minutos;

el pickle la reduce a ∼14 s por ejecución). Con ello la extracción completa de ambos modelos (97,760 vectores) corre estable con 1.9 GB libres.

5 Probes y estadística honesta

5.1 Diseño experimental

La familia de comparaciones está pre-registrada en config: 2 modelos × 4 capas × 2 modos × 2 tipos de probe = 32 configuraciones, sin ampliación posterior a la vista de resultados. Cada configuración se entrena en 4 folds y evalúa en el fold de salida, con el StandardScaler ajustado *dentro* del fold de entrenamiento (el scaler leakage filtra estadísticas del test y es sutil: el contrato lo fija y el punto de revisión de la Fase 3 lo verificó). La regresión logística selecciona $C \in \{0.01, 0.1, 1, 10\}$ por CV interna agrupada (3 folds por entidad) sobre los folds de entrenamiento, nunca tocando el fold de evaluación; el MLP de 64 unidades usa early stopping con semilla 42. Las 32 configuraciones producen 194,720 predicciones out-of-fold (6085 statements × 32) en `logs/oof_predictions.jsonl`, cada fila con su baseline $|\log p|$ para la comparación pareada.

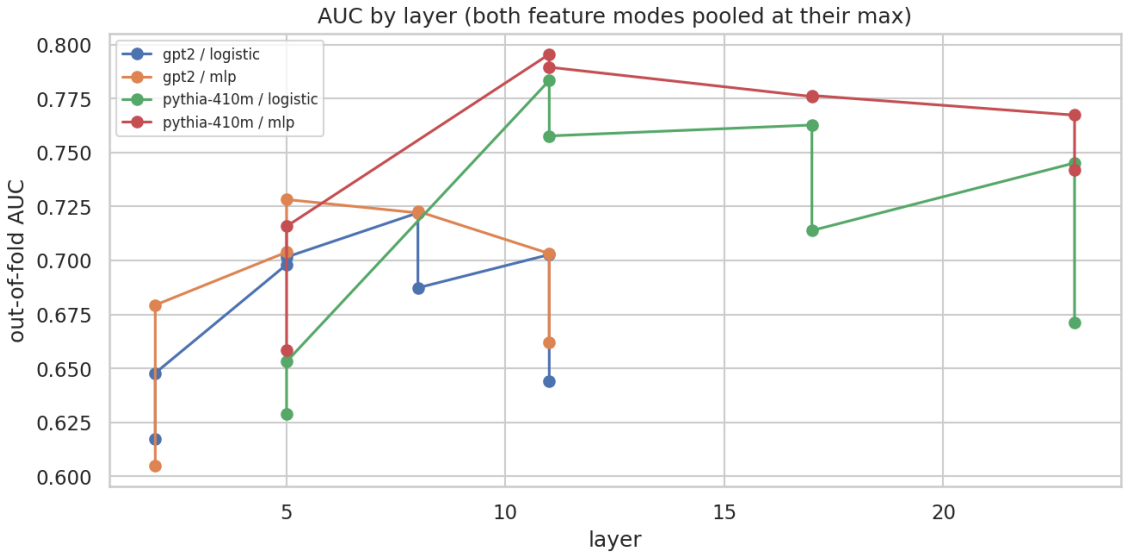


Figure 2: AUC out-of-fold por capa para ambos modelos (máximo sobre modos, por tipo de probe). El pico en capa media —gpt2 capa 5 de 12, pythia capa 11 de 24— y el declive hacia la última capa replican el patrón de la literatura.

5.2 Resultados del grid

Table 1: Mejor configuración por modelo y comparación pareada contra el baseline $|\log p|$ del propio modelo (1000 remuestreos pareados, misma partición). AUC con el ancla Mann-Whitney corregida; ver Sección **??**.

Modelo	Capa	Config	AUC [IC 95%]	AP	ECE	ΔAUC
gpt2	5	mean-pool MLP	**0.728** [0.716, 0.740]	0.737	0.061	+0.295 [+0.28, +0.32]
pythia-410m	11	last-token MLP	**0.796** [0.784, 0.807]	0.806	0.072	+0.396 [+0.38, +0.41]
baseline $\|\log p\|$	n/a	n/a	0.433 / 0.400	n/a	n/a	n/a

Los titulares de la Tabla 1 se sostienen en el detalle: el AUC de pythia-410m crece de 0.74 en capa 5 a 0.796 en capa 11 y cae a 0.768 en la capa 23; gpt2 hace el mismo arco (0.704 →

0.728 → 0.704). Las diferencias entre modos de feature son menores que las entre capas, y el MLP supera a la logística por ≤0.01 en las mejores configuraciones. La separabilidad geométrica que muestran los PCA del notebook 02 (Figura 3) es coherente con un probe lineal leyendo una dirección en el residual stream, no magnitudes. El baseline $|\log p|$ queda por debajo del azar (0.40–0.43): la magnitud de la sorpresa promedio no ordena la verdad, y por eso la diferencia pareada a favor del probe es grande y estable.

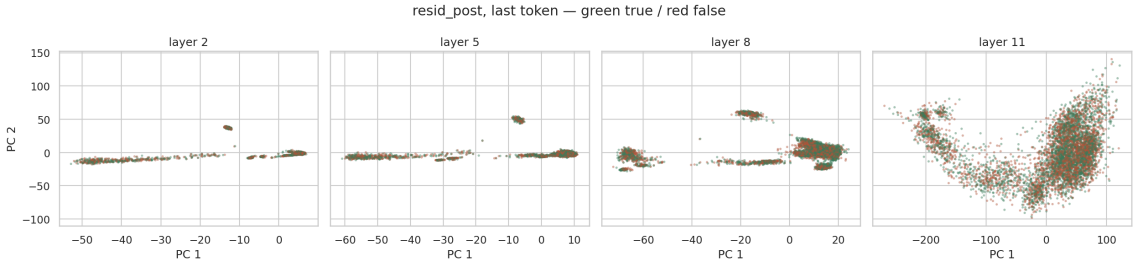


Figure 3: PCA (2 componentes) de la activación last-token de gpt2 por capa, coloreada por etiqueta (verde true, rojo false). La separabilidad crece hacia las capas medias: la firma geométrica que los probes lineales leen.

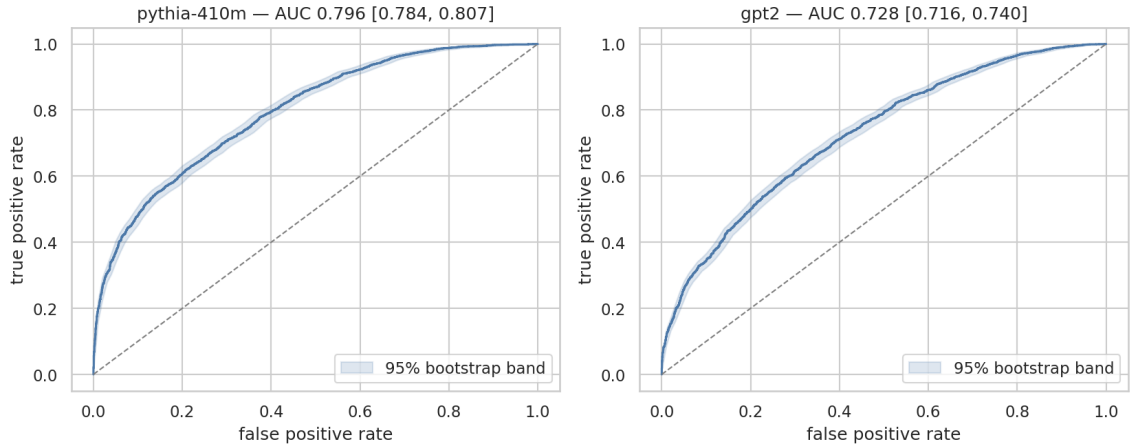


Figure 4: ROC de la mejor configuración por modelo con banda de bootstrap 95% (1000 remuestreos sobre una rejilla fija de FPR) y AUC con IC en la leyenda.

5.3 Controles y análisis secundarios

Confound estilístico. El peligro clásico de este dataset es que los statements false contengan negaciones: un probe que aprenda la palabra *not* reportaría AUC alto aprendiendo estilo, no verdad. El control entrena una logística sobre TF-IDF de 1–2 grams de los statements crudos con los mismos folds agrupados: **AUC 0.486 [0.472, 0.502]**, en el azar. En esta versión vendida del dataset el confound léxico simple no existe, y la brecha de 0.24–0.31 AUC entre el control y los probes neuronales es la evidencia de que la señal no es bag-of-words.

Pares mínimos. El control por pares emparejados (mismo sujeto, mismo template, solo cambia la verdad) detectó un único par en los ficheros vendidos; con $n = 1$ el subconjunto no tiene valor inferencial y se reporta como inconcluso en `logs/style_pairs.json`, nunca como evidencia.

Leave-one-domain-out. Entrenar en cinco dominios y evaluar en el sexto, para la mejor configuración de cada modelo, produce la caída honesta: gpt2 0.728 → 0.566 (rango por dominio

0.53–0.61) y pythia-410m 0.796 → 0.592 (0.57–0.61). Es exactamente la forma que la especificación anticipaba ("si el AUC in-domain es 0.80 y OOD es 0.55, eso es el resultado honesto") y acota el alcance de cualquier claim: el probe lee verdad *dentro* de la distribución de dominios que vio.

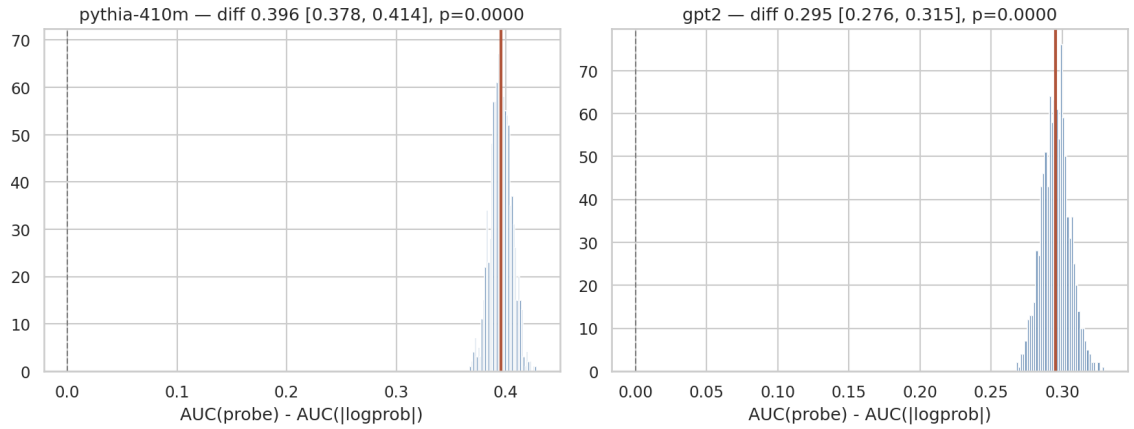


Figure 5: Bootstrap pareado de la diferencia AUC(probe) − AUC($|\log p|$) por modelo: los mismos índices remuestreados para ambos sistemas preservan su correlación. La distribución se aleja del cero con IC que lo excluye.

6 Generación, scoring y abstención

6.1 Pipeline de generación

El subset de TriviaQA (`rc.nocontext`) se construye por streaming determinista: primeras 20,000 preguntas con respuesta verificable en orden del split, muestra de 500 con semilla 42, cacheada en `data/processed/triviaqa_subset.jsonl` con sha256 en el manifest. La generación es greedy (temperatura 0.0) de 32 tokens por pregunta y por modelo, con transcripciones completas y reanudación por `prompt_id`. Nota de implementación documentada: la generación libre usa `generate()` de transformers con KV-cache (idénticos pesos, tokenizador y decodificación greedy, un orden de magnitud más rápido en CPU), mientras que extracción y scoring de probes corren estrictamente por los hooks de TransformerLens.

6.2 Ground truth estadístico y su auditoría

El scoring extrae la primera oración declarativa (las interrogativas se saltan; las generaciones sin oración declarativa se marcan y excluyen, política declarada una vez y aplicada uniformemente) y aplica string-match normalizado contra los aliases. La auditoría manual obligatoria del 10% detectó un defecto real del matching naívo: cuando la respuesta ya está contenida en la pregunta (frecuente en TriviaQA), *repetir* la pregunta cuenta como acierto. La corrección, documentada en el código, excluye y cuenta los casos donde todos los aliases coincidentes son subcadenas de la pregunta (estado `alias_in_question`), en lugar de forzarlos. Tras la corrección: gpt2 acierta **1.3%** y pythia-410m **6.6%** de las 500 preguntas, con 30 exclusions por modelo. Las cifras son consistentes con lo publicado para modelos pequeños en cerrado y se reportan como están.

6.3 Abstención selectiva y calibración

La curva cobertura-precisión barre umbrales τ respondiendo solo generaciones con score $\geq \tau$; τ^* maximiza la precisión sujeto a cobertura ≥ 0.6. Para gpt2, $\tau^* = 0.645$ produce (0.60, 0.018) frente a un piso de 0.013: ganancia +0.5 pp. Para pythia-410m, $\tau^* = 0.465$ produce (0.64,

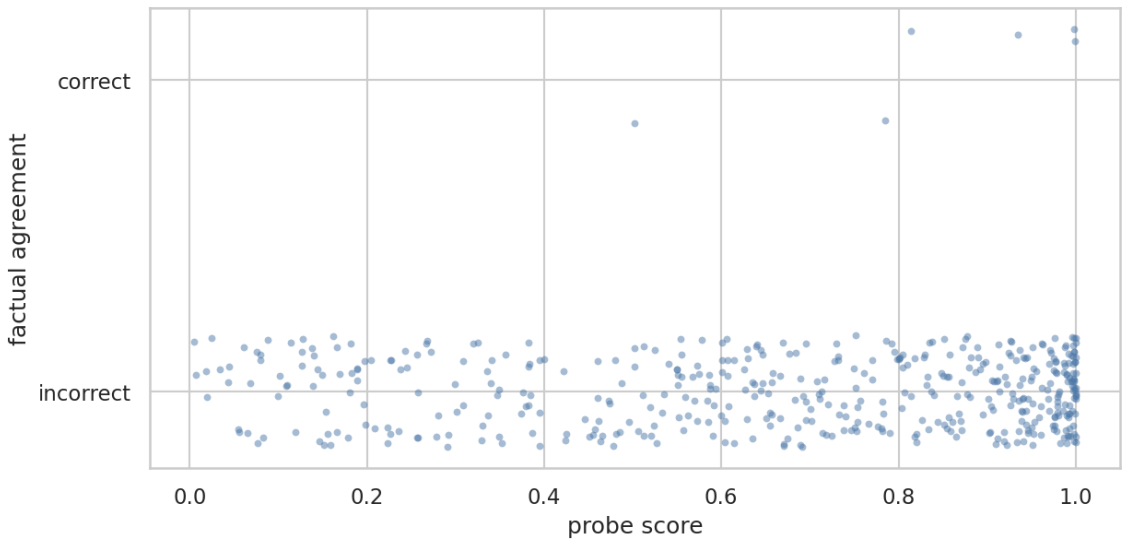


Figure 6: Score del probe (mejor configuración de cada modelo, reajustada sobre el dataset completo) contra acuerdo factual por generación (gpt2, con jitter vertical). El interés del análisis es si el score ordena los pocos aciertos por encima de las confabulaciones.

0.093) sobre piso 0.066: ganancia +2.7 pp. La calibración en generación es severa (ECE 0.66 y 0.52; Figura 8): los scores ordenan pero no son probabilidades, razón por la cual la política de abstención es rank-based por diseño. Los casos cualitativos del notebook 04 (confabulaciones capturadas con score alto y respuesta incorrecta, con texto completo de generación) son la cara visible de lo que el framework mide.

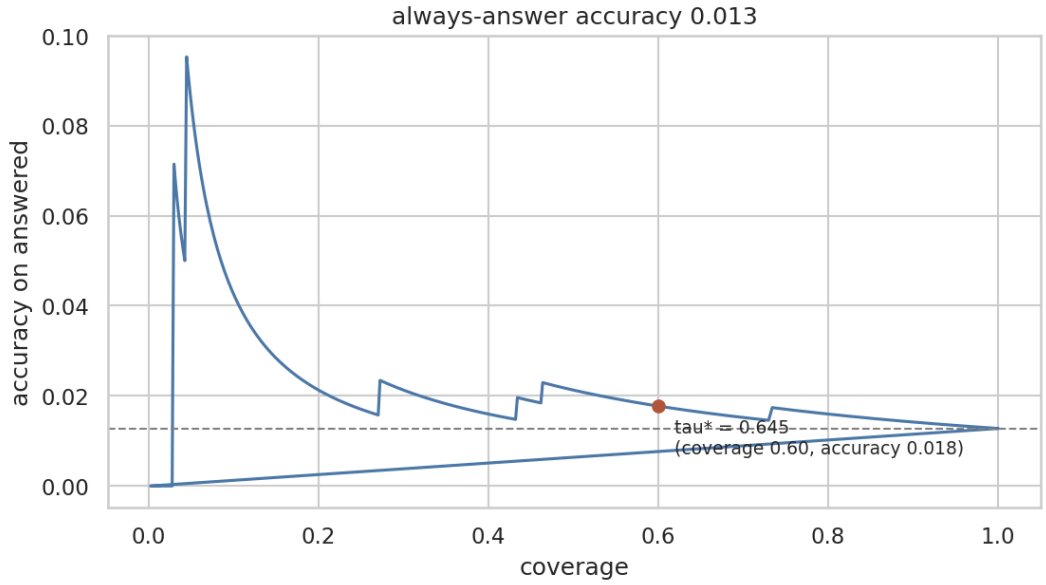


Figure 7: Curva cobertura-precisión con τ^* marcado (pythia-410m). La línea gris es la precisión de responder siempre; la ganancia operativa es la diferencia vertical en la cobertura objetivo.

7 Ensemble simbólico: disponible pero no activado

El ancla `config.yaml` fija `ensemble.prometheus_graph: null`, y el contrato es explícito: sin grafo, el ensemble se marca *unavailable* con su causa y el resto del pipeline se completa igual. El

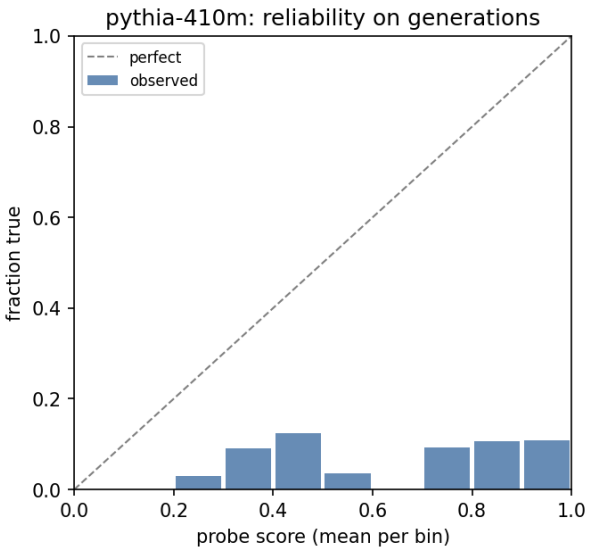


Figure 8: Diagrama de fiabilidad del score del probe sobre generaciones de pythia-410m (ECE 0.522, Brier 0.425): los scores están lejos de ser probabilidades utilizables, y así se declara en los límites.

registro producido (`logs/ensemble_results.json`) documenta la causa exacta. El puente, sin embargo, está implementado y probado: extracción de tripletas is-a de las oraciones generadas, señal simbólica $\{0, 0.5, 1\}$ por consistencia contra el grafo, y ensemble logístico de 2 features ajustado out-of-fold (5 folds, semilla 42) dentro del subconjunto extraíble, con comparación probe solo / señal sola / ensemble por bootstrap pareado en las mismas filas. Los tests con grafo sintético (`tests/test_ensemble.py`) fijan la extracción, la señal y la maquinaria del ensemble. Cablear PROMETHEUS-NS equivale a apuntar la ruta del grafo exportado en una línea de config y re-ejecutar `make ensemble`; el número que salga manda, y si el ensemble no mejora, eso se reporta.

8 Notebooks de demostración

Los cuatro notebooks se generan desde el productor único (`scripts/make_notebooks.py`, idempotente) y se ejecutan headless en orden por `scripts/run_notebooks.py` (nbclient, kernel fijado al intérprete del venv, celda fallida = target rojo). Todos ejecutaron sin errores y se commitearon con outputs (24, 18, 23 y 26 celdas). El contrato anti-hardcoding se cumple por diseño: las celdas leen de `data/processed`, `artifacts/` y `logs/`, las celdas guard reproducen lo que falte vía CLI idempotente, y las celdas markdown describen qué muestra cada figura y cómo leerla, nunca cifras. Las figuras se persisten a `docs/figures/` (dpi 150) vía `cassandra_ns.report.figures.save_figure`, de donde este informe toma las suyas.

Durante la primera ejecución headless se corrigieron tres defectos de reconstrucción en el productor, previos a la generación de outputs: las celdas del notebook 02 leían del fichero de resultados (como dicta la sección 8 de la especificación) en lugar de las predicciones crudas, los notebooks 02 y 04 usaban `_ensure` sin la celda guard que la define, y el notebook 03 referenciaba el dataframe de datos sin cargarlo. Además, `load_scored` convierte `correct` a boolean nullable para que la negación de pandas sea la Kleene correcta. Ninguna corrección alteró los contratos del ancla; todas quedaron cubiertas por la ejecución verde final.

9 Registro de riesgos: qué se materializó y cómo se blindó

URL del dataset viva (severidad: corrompe datos). Materializado y blindado. El repositorio original 404. Se ancló el espejo ApolloResearch a SHA completo, con manifest sha256 y fail-loud, y verificación cruzada contra un segundo espejo independiente. La instrucción de actualización del propio ancla cubría el caso.

Leakage por entidad (severidad: invalidante). Blindado por construcción. StratifiedGroupKFold por entidad, folds fijos con sha256, tests de cero leakage, gate de auditoría ante AUC > 0.95 (nunca activado: máximo 0.807 en el techo del IC) y leave-one-domain-out como análisis secundario obligatorio, ejecutado y reportado.

Confound de estilo (severidad: invalidante). Blindado y medido. Probe de control TF-IDF en el azar (0.486); pares emparejados insuficientes en el dataset ($n = 1$, reportado como inconcluso). La señal no es léxica en esta versión de los datos.

Scaler leakage (severidad: invalidante). Blindado por construcción. El StandardScaler se ajusta dentro de cada fold de entrenamiento; el contrato de `train.py` lo fija y la revisión de Fase 3 lo verificó antes del primer entrenamiento.

String-match de TriviaQA (severidad: corrompe datos). Materializado y blindado. La auditoría manual detectó el caso *respuesta contenida en la pregunta*; la corrección excluye y cuenta (`alias_in_question`) en lugar de forzar el acierto. Los umbrales del match rate (1.3% y 6.6%) se validaron contra la literatura de modelos pequeños en cerrado.

Presupuesto de memoria y velocidad (severidad: cuesta tiempo). Materializado y blindado. Tres OOM/determinaciones corregidas: filtro de hooks resid-only, logsumexp en vez de softmax completo, mmap en la recarga del modelo; pickle de la conversión TransformerLens (9 min → 14 s). El pipeline completo corre en 2 núcleos / 3.9 GB con reanudación idempotente.

Notebooks headless (severidad: cuesta tiempo). Materializado y blindado. Backend Agg, kernel registrado del venv, celdas guard con CLI idempotente, outputs <1 MB (dataframes con `.head()`). Primera ejecución roja por defectos de reconstrucción; corrección quirúrgica y ejecución verde final de los cuatro.

10 Limitaciones

Los resultados son **correlacionales**: los probes leen correlaciones del residual stream y nada aquí demuestra que el modelo "sabe" algo. El colapso OOD medido (0.73→0.57, 0.80→0.59) delimita el alcance in-distribution y replica la degradación documentada en la literatura reciente de probes de veracidad. Los modelos son pequeños (124M y 410M) y el suelo de acierto en TriviaQA cerrado (1.3% y 6.6%) hace que la ganancia de abstención sea un demo del framework, no una política de producción. El ground truth de generación es estadístico, no semántico: paráfroas fuera de la lista de aliases cuentan como incorrectas, y 30 generaciones por modelo quedaron excluidas por ambigüedad, nunca forzadas. Los conjuntos de prueba son modestos (6085 statements,

500 generaciones): los IC bootstrap tienen anchos de 0.01–0.05 y las diferencias dentro de esos anchos no se reclaman como hallazgos. Los scores del probe no son probabilidades calibradas en generación (ECE 0.52–0.66); el umbral de abstención es rank-based a propósito. Y la familia Holm está pre-registrada en config: los p-valores reportados corresponden exactamente a las 32 comparaciones registradas, sin expansiones post-hoc.

11 Reproducibilidad

El repositorio entrega 9 commits atómicos que siguen las fases de la especificación: esqueleto y anclas; contratos verdes en clon fresco; datos y folds; extracción con cache; corrección de memoria; probes con CV agrupada y estadística honesta; generación y abstención; ensemble; notebooks ejecutados; y reporte final con métricas medidas. `make test` corre 37 tests (estructura, splits, cache, ancla estadística, ensemble y regresión de ties del AUC) y queda verde tras cada fase. El entorno resuelto está congelado en `requirements.freeze.txt` (Python 3.10.21, torch 2.4.1+cpu, transformer-lens 2.15.0, transformers 4.46.3, scikit-learn 1.5.2, spaCy 3.8.16). La ruta rápida de reproducción es `make data && make extract && make probe && make report && make generate && make notebooks`; cada objetivo es idempotente, reanudable y falla ruidosamente ante desviaciones de integridad (sha256 del dataset, del manifest, de los folds y del cache se verifican en lectura). En Colab, la celda 1 de cualquier notebook clona, instala, descarga datos y deja los caches en `/content/artifacts`; la re-ejecución tras un reinicio del runtime continúa donde quedó, como documenta el propio notebook.